

Developer Operations, Monitoring and References Guide

Combined VS Code, AWS linking, monitoring and reference guide

Flipkart Clone Microservices Workspace

Developer Operations, Monitoring and References Guide

This is a combined guide for what to do after opening the Flipkart Clone project in VS Code, how to connect it with AWS infrastructure, how to monitor the deployment, and which official references to use.

Start After Code Is Opened in VS Code

This document explains exactly what to do after you have the full Flipkart Clone code opened in **Visual Studio Code**.

Workspace path:

```
/home/user/flipkart-clone
```

If you downloaded the ZIP, extract it first and open the extracted `flipkart-clone` folder in VS Code.

PART 1: Required Software

Install these on your computer:

Tool	Required For
VS Code	Editing and running the project
Node.js 18 or newer	Local development and frontend build
Docker Desktop / Docker Engine	Running MongoDB and all microservices
Docker Compose	Starting the full app with one command
Git	Version control and pushing to GitHub
AWS CLI v2	Pushing images and linking project to AWS
AWS Toolkit for VS Code	Viewing AWS resources inside VS Code
Terraform optional	Building AWS infrastructure from code

Recommended VS Code extensions:

- AWS Toolkit
- Docker
- ESLint
- Prettier
- HashiCorp Terraform
- REST Client
- GitLens

PART 2: Open the Project in VS Code

1. Open VS Code.
2. Click **File** → **Open Folder**.
3. Select the project folder:

```
/home/user/flipkart-clone
```

4. Open the integrated terminal:

```
Terminal → New Terminal
```

5. Confirm you are in the project root:

```
pwd  
ls
```

You should see:

```
api-gateway  
user-service  
product-service  
cart-service  
order-service  
payment-service  
notification-service  
frontend  
docker-compose.yml  
README.md  
HOW_TO_RUN.md
```

PART 3: First Run Using Docker Compose

This is the easiest and recommended way.

Step 3.1: Create Environment File

```
cp .env.example .env
```

Open `.env` and change `JWT_SECRET` to a long random value before real usage.

Example:

```
JWT_SECRET=my-long-random-secret-change-this  
FRONTEND_PORT=8080  
API_GATEWAY_PORT=3000  
MONGODB_PORT=27017
```

Step 3.2: Start All Services

```
docker compose up --build
```

If you want to run in background:

```
docker compose up --build -d
```

Step 3.3: Seed Products

Open a second terminal in VS Code and run:

```
docker compose exec product-service npm run seed
```

Step 3.4: Open Application

```
Frontend:    http://localhost:8080  
API Gateway: http://localhost:3000  
Health:      http://localhost:3000/health
```

Register a new user, browse products, add to cart, checkout and view orders.

PART 4: Useful Docker Commands in VS Code Terminal

Check running containers:

```
docker compose ps
```

View all logs:

```
docker compose logs -f
```

View one service log:

```
docker compose logs -f api-gateway
```

Restart one service:

```
docker compose restart product-service
```

Rebuild one service after code changes:

```
docker compose up -d --build product-service
```

Stop all services:

```
docker compose down
```

Stop and delete MongoDB data:

```
docker compose down -v
```

Run backend syntax check:

```
./scripts/check-backend-syntax.sh
```

Run frontend build check:

```
cd frontend  
npm install  
npm run build
```

PART 5: Local Development Mode Without Full Docker

Use this only if you want to edit and hot-reload individual services.

Step 5.1: Start MongoDB Only

```
docker run -d --name flipkart-dev-mongo -p 27017:27017 mongo:7
```

If container already exists:

```
docker start flipkart-dev-mongo
```

Step 5.2: Install Service Dependencies

Run this once for each service:

```
cd api-gateway && npm install
cd ../user-service && npm install
cd ../product-service && npm install
cd ../cart-service && npm install
cd ../order-service && npm install
cd ../payment-service && npm install
cd ../notification-service && npm install
cd ../frontend && npm install
```

Step 5.3: Start Services in Separate Terminals

Terminal 1:

```
cd user-service
npm run dev
```

Terminal 2:

```
cd product-service
npm run dev
```

Terminal 3:

```
cd cart-service
npm run dev
```

Terminal 4:

```
cd order-service
npm run dev
```

Terminal 5:

```
cd payment-service
npm run dev
```

Terminal 6:

```
cd notification-service
npm run dev
```

Terminal 7:

```
cd api-gateway  
npm run dev
```

Terminal 8:

```
cd frontend  
npm start
```

If React says port 3000 is already used, accept another port like 3007 because API Gateway uses 3000.

PART 6: Testing APIs from VS Code

Use terminal or REST Client extension.

Gateway health:

```
curl http://localhost:3000/health
```

Get products:

```
curl "http://localhost:3000/api/products?limit=5"
```

Register user:

```
curl -X POST http://localhost:3000/api/users/register \
-H "Content-Type: application/json" \
-d '{
  "name": "Test User",
  "email": "test@example.com",
  "password": "password123",
  "phone": "9876543210"
}'
```

Login user:

```
curl -X POST http://localhost:3000/api/users/login \
-H "Content-Type: application/json" \
-d '{
  "email": "test@example.com",
  "password": "password123"
}'
```

PART 7: Typical Developer Workflow

Use this flow every day:

1. Open VS Code.

2. Pull latest code:

```
git pull
```

3. Start app:

```
docker compose up -d
```

4. Make code changes.

5. Rebuild changed service:

```
docker compose up -d --build service-name
```

Example:

```
docker compose up -d --build product-service
```

6. Check logs:

```
docker compose logs -f product-service
```

7. Test in browser.

8. Commit changes:

```
git add .  
git commit -m "Update product service"  
git push
```

PART 8: Prepare Code for AWS Deployment

Before AWS deployment:

1. Make sure local Docker run works.
2. Seed data locally and test user/cart/order flow.
3. Build frontend successfully.
4. Push code to GitHub.
5. Create AWS ECR repositories.
6. Build and push Docker images to ECR.
7. Create ECS task definitions and services.
8. Update production environment variables.
9. Monitor ECS service logs.

Local checks:

```
./scripts/check-backend-syntax.sh  
cd frontend && npm install && npm run build
```

Docker checks:

```
docker compose up --build -d  
docker compose ps  
docker compose logs -f
```

PART 9: Common Problems and Fixes

Docker is not running

Start Docker Desktop or Docker Engine, then run:

```
docker info
```

Products are empty

Seed products:

```
docker compose exec product-service npm run seed
```

Frontend cannot call API

Check:

- API Gateway is running on port 3000.
- REACT_APP_API_URL is correct.
- Docker Compose frontend uses /api proxy.
- Browser console network tab.

Port already in use

Edit .env:

```
FRONTEND_PORT=8081
API_GATEWAY_PORT=3007
MONGODB_PORT=27018
```

Restart:

```
docker compose down
docker compose up --build
```

Service keeps restarting

Check logs:

```
docker compose logs -f service-name
```

Common causes:

- Wrong MongoDB URI
- Missing environment variable
- Syntax error
- Port conflict

PART 10: Minimum Commands You Need to Remember

```
cd /home/user/flipkart-clone
cp .env.example .env
docker compose up --build
```

Second terminal:

```
docker compose exec product-service npm run seed
```

Open:

```
http://localhost:8080
```

Link VS Code with AWS Infrastructure and Build Infra Guide

This guide explains how to connect your VS Code workspace to AWS, build Docker images, push them to ECR, deploy to ECS, and optionally create infrastructure from VS Code using Terraform.

Use this after your code is already opened in VS Code.

PART 1: What This Guide Helps You Do

You will learn how to:

- Connect VS Code terminal to your AWS account.
- Use AWS Toolkit extension in VS Code.
- Store AWS deployment variables in a local file.
- Build Docker images from VS Code.
- Push images to Amazon ECR.
- Force ECS services to deploy new images.
- Use Terraform from VS Code to build infrastructure if required.
- Connect GitHub Actions or CodePipeline for automated deployment.

PART 2: Install VS Code Extensions

Open VS Code extensions panel and install:

Extension	Purpose
AWS Toolkit	Connect VS Code with AWS account and resources
Docker	View and manage containers/images
HashiCorp Terraform	Terraform syntax and workflow
ESLint	JavaScript linting
Prettier	Code formatting
GitLens	Git history and blame
REST Client	Test APIs inside VS Code

The workspace also includes recommended extension settings in:

```
.vscode/extensions.json
```

PART 3: Configure AWS Access from VS Code Terminal

Option A: IAM Identity Center SSO Recommended

If your AWS account uses IAM Identity Center:

```
aws configure sso
```

Enter:

```
SSO session name: flipkart
SSO start URL: your AWS access portal URL
SSO region: us-east-1
CLI default client Region: us-east-1
CLI default output format: json
Profile name: flipkart-dev
```

Login:

```
aws sso login --profile flipkart-dev
```

Verify:

```
aws sts get-caller-identity --profile flipkart-dev
```

Option B: Access Keys

Use this only for development labs. Do not commit keys.

```
aws configure --profile flipkart-dev
```

Enter:

```
AWS Access Key ID
AWS Secret Access Key
Default region: us-east-1
Default output: json
```

Verify:

```
aws sts get-caller-identity --profile flipkart-dev
```

Make Profile Default in Current Terminal

Linux/macOS/Git Bash:

```
export AWS_PROFILE=flipkart-dev
export AWS_REGION=us-east-1
```

PowerShell:

```
$env:AWS_PROFILE="flipkart-dev"
$env:AWS_REGION="us-east-1"
```

Verify:

```
aws sts get-caller-identity
```


PART 4: Connect AWS Toolkit in VS Code

1. Open VS Code.
2. Click AWS icon in left sidebar.
3. Click **Connect to AWS**.
4. Choose your profile, for example `flipkart-dev`.
5. Select region `us-east-1`.
6. Use AWS Explorer to view resources.

Depending on extension version, you can view resources such as:

- ECR repositories
- ECS clusters and tasks
- CloudWatch Logs
- S3 buckets
- Lambda functions
- CloudFormation stacks

Use AWS Toolkit for quick navigation, but keep deployment commands scripted for repeatability.

PART 5: Create Local AWS Environment File

Create a local file from the template:

```
cp scripts/aws-env.example scripts/aws-env.local
```

Edit `scripts/aws-env.local`:

```
AWS_PROFILE=flipkart-dev  
AWS_REGION=us-east-1  
AWS_ACCOUNT_ID=123456789012  
ECS_CLUSTER=flipkart-cluster  
ALB_DNS_NAME=flipkart-alb-xxxx.us-east-1.elb.amazonaws.com
```

Never commit `scripts/aws-env.local` if it contains account-specific or sensitive values.

PART 6: ECR Build and Push from VS Code Terminal

Step 6.1: Confirm ECR Repositories Exist

```
aws ecr describe-repositories --region us-east-1
```

You should have:

```
flipkart-api-gateway
flipkart-user-service
flipkart-product-service
flipkart-cart-service
flipkart-order-service
flipkart-payment-service
flipkart-notification-service
flipkart-frontend
```

Step 6.2: Login to ECR

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com
```

Step 6.3: Build and Push One Service

Example for API Gateway:

```
cd api-gateway
docker build -t flipkart-api-gateway .
docker tag flipkart-api-gateway:latest ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-
gateway:latest
docker push ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/flipkart-api-gateway:latest
cd ..
```

Step 6.4: Build and Push All Services with Template Script

The workspace includes a helper template:

```
scripts/aws-ecr-build-push-template.sh
```

Usage:

```
cp scripts/aws-env.example scripts/aws-env.local
# edit scripts/aws-env.local first
./scripts/aws-ecr-build-push-template.sh
```

This builds and pushes all 8 Docker images.

PART 7: Deploy New Images to ECS from VS Code

After images are pushed to ECR, force ECS to start new tasks:

```
aws ecs update-service \  
  --cluster flipkart-cluster \  
  --service api-gateway-service \  
  --force-new-deployment \  
  --region us-east-1
```

Repeat for each service.

The workspace includes a helper template:

```
scripts/aws-ecs-force-deploy-template.sh
```

Run:

```
./scripts/aws-ecs-force-deploy-template.sh
```

Then monitor:

```
aws ecs describe-services \  
  --cluster flipkart-cluster \  
  --services api-gateway-service \  
  --region us-east-1
```

In AWS Console:

```
ECS → Clusters → flipkart-cluster → Services
```

PART 8: Build Infrastructure from VS Code

You have three choices.

Choice A: Console-Only Infrastructure

Use the PDF:

`docs/AWS_CONSOLE_ONLY_SETUP_GUIDE.pdf`

This is best for learning AWS visually.

Choice B: Terraform from VS Code

The workspace has a Terraform starter:

`infrastructure/terraform`

Commands:

```
cd infrastructure/terraform
terraform init
terraform fmt
terraform validate
terraform plan -var='docdb_master_password=ChangeThisStrongPassword123!'
terraform apply -var='docdb_master_password=ChangeThisStrongPassword123!'
```

This creates baseline infrastructure like:

- VPC
- Public/private subnets
- ECS cluster
- ECR repositories
- ALB
- Security groups
- DocumentDB starter

Destroy lab infra when done:

```
terraform destroy -var='docdb_master_password=ChangeThisStrongPassword123!'
```

Warning: Terraform destroy deletes infrastructure. Use only in test/lab environments.

Choice C: CI/CD Builds Infrastructure or Deploys App

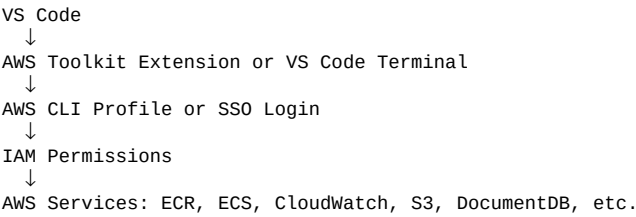
Use one of:

- AWS CodePipeline + CodeBuild
- GitHub Actions
- Jenkins on EC2

For most students/projects, GitHub Actions is easiest.

PART 9: How VS Code Links to Your AWS Infra

VS Code does not magically connect to AWS infrastructure by itself. It connects through these layers:



The important files are:

File	Purpose
.env	Local Docker Compose environment
scripts/aws-env.local	Local AWS account/deploy variables
docker-compose.yml	Local full-stack run
Dockerfile in each service	Image build instructions
infrastructure/terraform/*.tf	Infrastructure as code
.github/workflows/ci.yml	CI syntax checks

PART 10: Recommended Deployment Flow from VS Code

Use this flow after AWS infrastructure is already created:

1. Confirm local app works:

```
docker compose up --build
```

2. Build frontend:

```
cd frontend
npm install
npm run build
cd ..
```

3. Check backend syntax:

```
./scripts/check-backend-syntax.sh
```

4. Login to AWS:

```
aws sso login --profile flipkart-dev
# or use aws configure profile
```

5. Build and push images:

```
./scripts/aws-ecr-build-push-template.sh
```

6. Force ECS deployment:

```
./scripts/aws-ecs-force-deploy-template.sh
```

7. Watch deployment:

```
AWS Console → ECS → flipkart-cluster → Services
AWS Console → EC2 → Target Groups → Targets
AWS Console → CloudWatch → Logs
```

8. Test app:

```
curl http://ALB_DNS_NAME/api/health
curl http://ALB_DNS_NAME/api/products
```

PART 11: VS Code Tasks

The workspace includes VS Code task definitions in:

`.vscode/tasks.json`

Useful tasks:

- Docker Compose Up
- Docker Compose Down
- Seed Products
- Backend Syntax Check
- Frontend Build
- Terraform Init
- Terraform Validate
- Terraform Plan

Run tasks:

Terminal → Run Task

PART 12: GitHub Actions Connection

If code is pushed to GitHub, CI/CD can build and deploy automatically.

Required GitHub Secrets

Add in GitHub:

Settings → Secrets and variables → Actions

Secrets:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION=us-east-1
AWS_ACCOUNT_ID
```

Better production approach:

- Use GitHub OIDC with IAM role instead of long-lived access keys.

Workflow Concept

```
Push code to GitHub
↓
GitHub Actions builds Docker image
↓
Pushes image to ECR
↓
Updates ECS service
↓
ECS starts new tasks
```

PART 13: CodePipeline Connection

If using AWS CodePipeline:

```
GitHub or CodeCommit
↓
CodePipeline Source Stage
↓
CodeBuild Docker Build
↓
ECR Push
↓
ECS Deploy
```

Use this if you want AWS-native CI/CD.

PART 14: Security Rules for VS Code and AWS

Never commit these:

- AWS access keys
- .env
- scripts/aws-env.local
- DocumentDB password
- JWT secret
- SMTP password
- Payment keys

Use:

- AWS Secrets Manager
- GitHub Actions secrets
- IAM Identity Center SSO
- IAM roles for ECS tasks

Check before commit:

```
git status
git diff --cached
```

PART 15: Common AWS CLI Commands from VS Code

Who am I?

```
aws sts get-caller-identity
```

List ECR repositories:

```
aws ecr describe-repositories --region us-east-1
```

List ECS clusters:

```
aws ecs list-clusters --region us-east-1
```

List ECS services:

```
aws ecs list-services --cluster flipkart-cluster --region us-east-1
```

Describe service:

```
aws ecs describe-services \
  --cluster flipkart-cluster \
  --services api-gateway-service \
  --region us-east-1
```

View recent CloudWatch log streams:

```
aws logs describe-log-streams \
  --log-group-name /ecs/flipkart-api-gateway \
  --order-by LastEventTime \
  --descending \
  --max-items 5 \
  --region us-east-1
```

PART 16: If You Want Fully Automated Infra + App Deployment

Recommended mature setup:

1. Terraform creates VPC, ECR, ECS, ALB, DocumentDB and security groups.
2. GitHub Actions builds Docker images and pushes to ECR.
3. GitHub Actions updates ECS services.
4. CloudWatch monitors and alerts.
5. Secrets Manager stores credentials.
6. Route 53 and ACM provide domain and HTTPS.

Workflow:

VS Code → Git commit → GitHub push → CI/CD → AWS ECS deployment

This is the cleanest long-term workflow.

PART 17: Final Checklist

- ☐ VS Code opened at project root.
- ☐ AWS Toolkit installed.
- ☐ AWS CLI configured with `flipkart-dev` profile.
- ☐ Docker is running.
- ☐ Local app runs with Docker Compose.
- ☐ ECR repositories exist.
- ☐ `scripts/aws-env.local` configured.
- ☐ Docker images pushed to ECR.
- ☐ ECS task definitions use ECR image URIs.
- ☐ ECS services running.
- ☐ ALB target groups healthy.
- ☐ CloudWatch logs visible.
- ☐ Application opens from ALB DNS or custom domain.

Monitoring and Operations Guide

This guide explains how to monitor the Flipkart Clone locally and on AWS after deployment.

It covers:

- Local Docker monitoring
- ECS service monitoring
- CloudWatch logs and metrics
- ALB and target group health
- DocumentDB monitoring
- CloudWatch dashboards
- CloudWatch alarms
- Logs Insights queries
- Incident troubleshooting runbooks
- Daily and weekly operations checklist

PART 1: Monitoring Locally with Docker

Check Container Status

```
cd /home/user/flipkart-clone
docker compose ps
```

All services should show running or healthy.

View All Logs

```
docker compose logs -f
```

View One Service Log

```
docker compose logs -f api-gateway
```

Other examples:

```
docker compose logs -f product-service
docker compose logs -f user-service
docker compose logs -f mongodb
```

Check Health Endpoints

Service	Local Health URL
API Gateway	http://localhost:3000/health
User Service	http://localhost:3001/health
Product Service	http://localhost:3002/health
Cart Service	http://localhost:3003/health
Order Service	http://localhost:3004/health
Payment Service	http://localhost:3005/health
Notification Service	http://localhost:3006/health
Frontend	http://localhost:8080/healthz

Test quickly:

```
curl http://localhost:3000/health
curl http://localhost:3000/api/products?limit=3
```

Local Smoke Test

```
./scripts/smoke-test.sh
```

PART 2: AWS Monitoring Overview

Production monitoring should include these layers:

Layer	AWS Service
Container logs	CloudWatch Logs
CPU and memory	ECS CloudWatch Metrics
Request/latency/5XX	Application Load Balancer Metrics
Database	DocumentDB Metrics
Service health	ECS Service Events and Target Groups
Dashboards	CloudWatch Dashboards
Alerts	CloudWatch Alarms + SNS
Tracing	AWS X-Ray optional
Security traffic	WAF logs optional
Cost	AWS Budgets and Cost Explorer

PART 3: ECS Service Monitoring

Where to Check ECS Health

1. Open AWS Console.
2. Go to **ECS**.
3. Open cluster: `flipkart-cluster`.
4. Open **Services** tab.
5. Check each service:
 - `frontend-service`
 - `api-gateway-service`
 - `user-service`
 - `product-service`
 - `cart-service`
 - `order-service`
 - `payment-service`
 - `notification-service`

What Healthy Looks Like

For each service:

Field	Healthy Value
Desired tasks	1 or 2 depending setup
Running tasks	Same as desired
Pending tasks	0
Failed deployments	0
Latest deployment	Completed or steady state

ECS Service Events

Open each ECS service and check **Events**.

Important error messages:

Error	Meaning
<code>CannotPullContainerError</code>	ECR image, tag, or IAM issue
<code>Essential container exited</code>	App crashed after startup
<code>Health checks failed</code>	ALB or app health endpoint issue
<code>ResourceInitializationError</code>	Secrets/logs/network issue
<code>Cannot resolve host</code>	Cloud Map or DNS issue

PART 4: CloudWatch Logs

Recommended Log Groups

Create or confirm these log groups:

```
/ecs/flipkart-api-gateway
/ecs/flipkart-user-service
/ecs/flipkart-product-service
/ecs/flipkart-cart-service
/ecs/flipkart-order-service
/ecs/flipkart-payment-service
/ecs/flipkart-notification-service
/ecs/flipkart-frontend
```

Set Log Retention

For development:

7 days

For production:

30 to 90 days

Steps:

1. Open **CloudWatch**.
2. Go to **Logs** → **Log groups**.
3. Select log group.
4. Click **Actions** → **Edit retention setting**.
5. Choose retention period.

Logs Insights Queries

Find Errors

```
fields @timestamp, @message
| filter @message like /ERROR|Error|error|failed|Failed/
| sort @timestamp desc
| limit 100
```

API Gateway 502 or Proxy Problems

```
fields @timestamp, @message
| filter @message like /Proxy error|Bad gateway|ECONNREFUSED|ETIMEDOUT/
| sort @timestamp desc
| limit 100
```

MongoDB Connection Problems

```
fields @timestamp, @message
| filter @message like /MongoDB|Mongoose|connection|ECONNREFUSED|authentication failed/
| sort @timestamp desc
| limit 100
```

Authentication Problems

```
fields @timestamp, @message
| filter @message like /Invalid token|Authentication|JWT|401/
| sort @timestamp desc
| limit 100
```

Slow Requests if Logs Include Timing

```
fields @timestamp, @message
| filter @message like /response time|duration|slow/
| sort @timestamp desc
| limit 50
```

PART 5: Application Load Balancer Monitoring

Where to Check

1. Go to **EC2**.
2. Click **Load Balancers**.
3. Select `flipkart-alb`.
4. Check tabs:
 - Monitoring
 - Listeners
 - Security
 - Target groups

Important ALB Metrics

Metric	Watch For
RequestCount	Traffic volume
TargetResponseTime	API/frontend latency
HTTPCode_ELB_5XX_Count	Load balancer errors
HTTPCode_Target_5XX_Count	App/service errors
HTTPCode_Target_4XX_Count	Bad requests/auth errors
HealthyHostCount	Healthy ECS tasks
UnHealthyHostCount	Unhealthy ECS tasks

Recommended ALB Alarm Thresholds

Alarm	Threshold
ALB 5XX	> 10 in 5 minutes
Target 5XX	> 20 in 5 minutes
Response time	> 2 seconds for 5 minutes
Unhealthy targets	> 0 for 2 periods

Target Group Health

Open **EC2** → **Target Groups**:

- `flipkart-api-tg`
- `flipkart-frontend-tg`

Targets should be **healthy**.

If unhealthy, check:

- Container port mapping
- Health check path

- Security group from ALB to service
- ECS task logs
- App is listening on correct port

PART 6: DocumentDB Monitoring

Open **DocumentDB** → `flipkart-docdb-cluster` → **Monitoring**.

Key Metrics

Metric	Meaning	Alert When
CPUUtilization	Database CPU	> 75-80%
DatabaseConnections	Active connections	Sudden spikes or near max
FreeableMemory	Available memory	Very low for sustained period
ReadThroughput	Read traffic	Unexpected spikes
WriteThroughput	Write traffic	Unexpected spikes
ReadLatency	Read delay	Increasing over baseline
WriteLatency	Write delay	Increasing over baseline
DiskQueueDepth	Disk pressure	Sustained high value

Common DocumentDB Problems

Too Many Connections

Fix:

- Use connection pooling.
- Ensure each service creates one Mongoose connection, not one per request.
- Scale database instance if needed.

Slow Product Search

Fix:

- Add indexes.
- Cache frequent reads in Redis.
- Avoid unbounded queries.

Authentication Failed

Fix:

- Verify username/password.
- Check Secrets Manager value.
- Recreate ECS task with updated secret.

PART 7: CloudWatch Dashboard Setup

Create dashboard:

1. Open **CloudWatch**.
2. Go to **Dashboards**.
3. Click **Create dashboard**.
4. Name: Flipkart-Monitoring.

Recommended Widgets

ECS CPU Widget

Metrics:

ECS → ClusterName, ServiceName → CPUUtilization

Include all services.

ECS Memory Widget

Metrics:

ECS → ClusterName, ServiceName → MemoryUtilization

ALB Traffic Widget

Metrics:

ApplicationELB → RequestCount
ApplicationELB → TargetResponseTime
ApplicationELB → HTTPCode_Target_5XX_Count
ApplicationELB → HTTPCode_ELB_5XX_Count

Target Health Widget

Metrics:

HealthyHostCount
UnHealthyHostCount

DocumentDB Widget

Metrics:

CPUUtilization
DatabaseConnections
FreeableMemory
ReadLatency
WriteLatency

PART 8: CloudWatch Alarms

Create SNS topic first:

```
flipkart-alerts
```

Subscribe your email and confirm the email subscription.

Recommended Alarms

Alarm Name	Metric	Condition
API-Gateway-High-CPU	ECS CPU	> 80% for 5 min
Product-Service-High-Memory	ECS Memory	> 85% for 5 min
ALB-Target-5XX-High	Target 5XX	> 20 in 5 min
ALB-Unhealthy-Targets	UnHealthyHostCount	> 0 for 2 periods
DocumentDB-High-CPU	DocDB CPU	> 80% for 10 min
DocumentDB-Low-Memory	FreeableMemory	Below safe baseline
CloudWatch-Log-Errors	Metric filter	Error count > threshold

Create Metric Filter for Errors

1. Open a CloudWatch log group.
2. Click **Metric filters**.
3. Create filter pattern:

```
?ERROR ?Error ?error ?failed ?Failed
```

4. Create metric namespace:

```
FlipkartApp
```

5. Metric name:

```
ServiceErrorCount
```

6. Create alarm on this metric.

PART 9: Deployment Monitoring

When deploying a new version:

1. Open ECS service.
2. Click **Deployments and events**.
3. Watch new tasks start.
4. Confirm old tasks drain.
5. Confirm target groups become healthy.
6. Watch CloudWatch logs.
7. Test health endpoints.
8. Test frontend in browser.

Deployment Verification Commands

From your local machine:

```
curl http://ALB_DNS_NAME/api/health  
curl http://ALB_DNS_NAME/api/products?limit=3
```

If HTTPS/domain is configured:

```
curl https://yourdomain.com/api/health  
curl https://yourdomain.com/api/products?limit=3
```

PART 10: Incident Runbooks

Runbook A: Frontend Not Opening

Check in order:

1. ALB DNS opens or not.
2. Frontend target group healthy.
3. Frontend ECS tasks running.
4. Frontend service logs.
5. Security group allows ALB to frontend port 80.
6. Nginx health endpoint `/healthz`.
7. Browser console errors.

Runbook B: API Returns 502

Check:

1. API target group health.
2. API Gateway ECS tasks running.
3. API Gateway logs.
4. API Gateway env vars point to correct Cloud Map names.
5. Backend services are running.
6. Security groups allow API Gateway to services.
7. Cloud Map DNS namespace is correct.

Runbook C: Login/Register Fails

Check:

1. User service logs.
2. DocumentDB connectivity.
3. JWT secret present.
4. MongoDB URI has correct database name.
5. API request body is valid JSON.
6. User collection has unique email index.

Runbook D: Products Not Showing

Check:

1. Product service running.
2. Product service logs.
3. Product database has seeded products.

4. Product route is `/api/products`.
5. Frontend API base URL is correct.

If database is empty, seed products locally before image build or create an admin seed task.

Runbook E: Cart Fails

Check:

1. Cart service logs.
2. Product service reachable from cart service.
3. `PRODUCT_SERVICE_URL` env var.
4. JWT token is present in request.
5. User ID matches logged-in user.

Runbook F: Order Checkout Fails

Check:

1. Order service logs.
2. Cart has items.
3. Cart service reachable from order service.
4. Notification service reachable.
5. Payment service logs.
6. DocumentDB connection for orders and payments.

PART 11: Cost Monitoring

Use:

- AWS Budgets
- Cost Explorer
- CloudWatch log retention
- Tags

Recommended tags:

```
Project=FlipkartClone
Environment=Dev or Prod
Owner=YourName
Service=api-gateway or user-service etc.
```

High-cost areas to watch:

Service	Why Cost Rises
NAT Gateway	Hourly cost and data transfer
DocumentDB	Instance hours and backups
CloudWatch Logs	High log ingestion and long retention
Fargate	Too many tasks or large CPU/memory
WAF	Paid managed rules or high request volume
DataDog	Per-host/APM pricing

PART 12: Daily Operations Checklist

Do this daily in production:

- ☐ Check ECS services all running.
- ☐ Check ALB target groups healthy.
- ☐ Check CloudWatch alarms state.
- ☐ Review last 24 hours error logs.
- ☐ Check DocumentDB CPU and connections.
- ☐ Check deployment events if any release happened.
- ☐ Check AWS Budget status.

PART 13: Weekly Operations Checklist

Do this weekly:

- ☐ Review CloudWatch dashboard trends.
- ☐ Review slow endpoints and high latency.
- ☐ Confirm backups are working.
- ☐ Check ECR old image cleanup policy.
- ☐ Check IAM users and keys.
- ☐ Rotate secrets if needed.
- ☐ Review WAF blocked requests.
- ☐ Review Cost Explorer by service.
- ☐ Confirm log retention settings.

PART 14: Production Monitoring Minimum Setup

If you want a minimum production setup, configure these at least:

1. ECS Container Insights.
2. CloudWatch log groups for every service.
3. ALB 5XX alarm.
4. ALB unhealthy target alarm.
5. ECS CPU and memory alarms.
6. DocumentDB CPU and connection alarms.
7. SNS email alert topic.
8. AWS Budget alert.
9. CloudWatch dashboard.

This gives enough visibility to operate the application safely.

References and Official Documentation

This document lists references for the Flipkart Clone project, AWS deployment, VS Code integration, Docker, monitoring and CI/CD.

Use official documentation as the source of truth because AWS console screens and pricing can change.

PART 1: Project Documents in This Workspace

Document	Purpose
README.md	Main project overview and quick start
HOW_TO_RUN.md	Local run instructions
docs/ARCHITECTURE.md	Project architecture notes
docs/AWS_CONSOLE_ONLY_SETUP_GUIDE.pdf	Complete AWS console deployment guide
docs/START_AFTER_CODE_IN_VSCODE.md	What to do after opening code in VS Code
docs/MONITORING_AND_OPERATIONS_GUIDE.md	Monitoring, alarms, dashboards and runbooks
docs/VSCODE_AWS_INFRA_AND_BUILD_GUIDE.md	Link VS Code with AWS infra, build/push/deploy
docs/REFERENCES.md	This references document

PART 2: AWS Core References

Topic	Official Reference
AWS Console	https://aws.amazon.com/console/
AWS Documentation Home	https://docs.aws.amazon.com/
AWS Well-Architected Framework	https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html
AWS Pricing Calculator	https://calculator.aws/
AWS Free Tier	https://aws.amazon.com/free/
AWS Regions and AZs	https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html

PART 3: IAM and Security References

Topic	Official Reference
IAM User Guide	https://docs.aws.amazon.com/IAM/latest/UserGuide/
IAM Best Practices	https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html
IAM Identity Center	https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html
IAM Roles	https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
AWS Secrets Manager	https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html
AWS KMS	https://docs.aws.amazon.com/kms/latest/developerguide/overview.html
AWS WAF	https://docs.aws.amazon.com/waf/latest/developerguide/what-is-aws-waf.html
AWS Shield	https://docs.aws.amazon.com/waf/latest/developerguide/shield-chapter.html
ACM Certificates	https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html

PART 4: Network and Load Balancing References

Topic	Official Reference
Amazon VPC	https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html
VPC Security Groups	https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
VPC Route Tables	https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html
NAT Gateways	https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
Elastic Load Balancing	https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/what-is-load-balancing.html
Application Load Balancer	https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html
ALB Target Groups	https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html
Route 53	https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/Welcome.html
CloudFront	https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html

PART 5: Container and Microservices References

Topic	Official Reference
Amazon ECS	https://docs.aws.amazon.com/AmazonECS/latest/developer-guide/welcome.html
AWS Fargate	https://docs.aws.amazon.com/AmazonECS/latest/developer-guide/AWS_Fargate.html
ECS Task Definitions	https://docs.aws.amazon.com/AmazonECS/latest/developer-guide/task_definitions.html
ECS Services	https://docs.aws.amazon.com/AmazonECS/latest/developer-guide/ecs_services.html
ECS Service Auto Scaling	https://docs.aws.amazon.com/AmazonECS/latest/developer-guide/service-auto-scaling.html
Amazon ECR	https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html
ECR Image Scanning	https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html
AWS Cloud Map	https://docs.aws.amazon.com/cloud-map/latest/dg/what-is-cloud-map.html
Docker Docs	https://docs.docker.com/
Docker Compose	https://docs.docker.com/compose/

PART 6: Database and Cache References

Topic	Official Reference
Amazon DocumentDB	https://docs.aws.amazon.com/documentdb/latest/developerguide/what-is.html
DocumentDB Connections	https://docs.aws.amazon.com/documentdb/latest/developerguide/connect_programmatically.html
DocumentDB Best Practices	https://docs.aws.amazon.com/documentdb/latest/developerguide/best_practices.html
MongoDB Manual	https://www.mongodb.com/docs/manual/
Mongoose Docs	https://mongoosejs.com/docs/
Amazon ElastiCache	https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/WhatIs.html
ElastiCache for Redis	https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/GettingStarted.html
Redis Docs	https://redis.io/docs/latest/

PART 7: Monitoring and Observability References

Topic	Official Reference
Amazon CloudWatch	https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html
CloudWatch Logs	https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/WhatIsCloudWatchLogs.html
CloudWatch Logs Insights	https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/AnalyzingLogData.html
CloudWatch Alarms	https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html
ECS Container Insights	https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/ContainerInsights.html
AWS X-Ray	https://docs.aws.amazon.com/xray/latest/devguide/aws-xray.html
ALB CloudWatch Metrics	https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-cloudwatch-metrics.html
DocumentDB Monitoring	https://docs.aws.amazon.com/documentdb/latest/developerguide/cloud_watch.html
AWS Budgets	https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html
Cost Explorer	https://docs.aws.amazon.com/cost-management/latest/userguide/ce-what-is.html

PART 8: CI/CD References

Topic	Official Reference
AWS CodePipeline	https://docs.aws.amazon.com/codepipeline/latest/userguide/welcome.html
AWS CodeBuild	https://docs.aws.amazon.com/codebuild/latest/userguide/welcome.html
CodeBuild Buildspec	https://docs.aws.amazon.com/codebuild/latest/userguide/build-spec-ref.html
ECS Deploy Action in CodePipeline	https://docs.aws.amazon.com/codepipeline/latest/userguide/action-reference-ECS.html
GitHub Actions	https://docs.github.com/en/actions
GitHub Actions for Amazon ECS	https://github.com/aws-actions/amazon-ecs-deploy-task-definition
AWS Credentials GitHub Action	https://github.com/aws-actions/configure-aws-credentials
Jenkins Documentation	https://www.jenkins.io/doc/
Jenkins Pipeline	https://www.jenkins.io/doc/book/pipeline/

PART 9: VS Code and Developer Tools References

Topic	Official Reference
Visual Studio Code	https://code.visualstudio.com/docs
AWS Toolkit for VS Code	https://docs.aws.amazon.com/toolkit-for-vscode/latest/userguide/welcome.html
VS Code Docker Extension	https://code.visualstudio.com/docs/containers/overview
VS Code Integrated Terminal	https://code.visualstudio.com/docs/terminal/basics
VS Code Tasks	https://code.visualstudio.com/docs/editor/tasks
Terraform Extension	https://marketplace.visualstudio.com/items?itemName=HashiCorp.terraform
REST Client Extension	https://marketplace.visualstudio.com/items?itemName=humao.rest-client

PART 10: Frontend and Backend Framework References

Topic	Official Reference
Node.js	https://nodejs.org/en/docs
Express.js	https://expressjs.com/
React	https://react.dev/
React Router	https://reactrouter.com/
Axios	https://axios-http.com/docs/intro
Nginx	https://nginx.org/en/docs/
JWT	https://jwt.io/introduction
bcrypt	https://www.npmjs.com/package/bcryptjs

PART 11: Infrastructure as Code References

Topic	Official Reference
Terraform	https://developer.hashicorp.com/terraform/docs
Terraform AWS Provider	https://registry.terraform.io/providers/hashicorp/aws/latest/docs
Terraform ECS Resources	https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/ecs_cluster
Terraform ECR Resources	https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/ecr_repository
Terraform VPC Resources	https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/vpc
Terraform DocumentDB Resources	https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/docdb_cluster

PART 12: Important Project-Specific Routes

The completed project uses these public API routes through API Gateway:

Feature	Route
Gateway health	/api/health or /health
Register	/api/users/register
Login	/api/users/login
Profile	/api/users/profile
Products	/api/products
Product detail	/api/products/:id
Cart	/api/cart/:userId
Orders	/api/orders
User orders	/api/orders/user/:userId
Payments	/api/payments
Notifications	/api/notifications/user/:userId

Important correction:

Use /api/products
Do not use /api/products/products

PART 13: Pricing References

Always check live AWS pricing before running production workloads:

Service	Pricing Page
ECS Fargate	https://aws.amazon.com/fargate/pricing/
DocumentDB	https://aws.amazon.com/documentdb/pricing/
Application Load Balancer	https://aws.amazon.com/elasticloadbalancing/pricing/
NAT Gateway	https://aws.amazon.com/vpc/pricing/
ECR	https://aws.amazon.com/ecr/pricing/
CloudWatch	https://aws.amazon.com/cloudwatch/pricing/
ElastiCache	https://aws.amazon.com/elasticache/pricing/
CloudFront	https://aws.amazon.com/cloudfront/pricing/
WAF	https://aws.amazon.com/waf/pricing/
Secrets Manager	https://aws.amazon.com/secrets-manager/pricing/

PART 14: Recommended Learning Order

If you are learning this end-to-end, follow this order:

1. Run the app locally with `HOW_TO_RUN.md`.
2. Open and understand `README.md`.
3. Read `docs/ARCHITECTURE.md`.
4. Follow `docs/START_AFTER_CODE_IN_VSCODE.md`.
5. Follow `docs/AWS_CONSOLE_ONLY_SETUP_GUIDE.pdf`.
6. Use `docs/VSCODE_AWS_INFRA_AND_BUILD_GUIDE.md` to connect local code to AWS.
7. Use `docs/MONITORING_AND_OPERATIONS_GUIDE.md` after deployment.
8. Use this references document when you need official docs.